Implementation Foundation

Campus Equipment Checkout System | Raghuram Kalasani | COMP-699-A

Step 2, Week 1 - corrected software development milestone, October 27, 2026

Purpose and correction

This milestone documents the initial executable software foundation for the software development project established in Step 1. The earlier research_foundation report followed the research-project instructions by mistake. This report supersedes that document for Week 1. The earlier research material is retained as historical work and is not evidence that the application requirements are complete.

Implemented source tree

```
src/campus_checkout/
  manage.py                 Django command entry point
  requirements.txt          Pinned application dependencies
  config/
    settings.py             Apps, SQLite, middleware, timezone
    urls.py                 Root redirect and admin route
    asgi.py, wsgi.py         Server entry points
  inventory/
    models.py               Equipment data model
    admin.py                Inventory interface and controls
    migrations/0001_initial.py
    tests.py                Six automated test methods
```

Components and interaction

The browser uses Django admin at /admin/. Django authentication and permissions protect that interface. Inventory forms validate input and use the Equipment model through the Django ORM. SQLite stores the equipment records. The root URL redirects to /admin/. The application is a local classroom prototype with a working inventory interface, not the complete lending system.

Equipment has a unique asset_tag, name, category, optional description, and is_active flag. Asset tags accept uppercase letters, digits, and hyphens. The database enforces uniqueness. The admin supports search and filtering, creation, editing, and retirement by clearing Active. Admin deletion is disabled to preserve records for future history.

Installation and execution

Run these commands from the repository root using Python 3.12. The tested environment used Python 3.12.14, Django 5.2.18, SQLite 3.53.1, and macOS 15.6.1 on arm64. Dependencies are pinned in src/campus_checkout/requirements.txt. Python 3.9 from the Mac system installation is not the interpreter used for this Django application.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r src/campus_checkout/requirements.txt
cd src/campus_checkout
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Choose a local demonstration username and password when prompted. No live account or database is included. If python3.12 is unavailable, use the path to an installed Python 3.12 interpreter for the first command. A previously working Python 3.12 virtual environment can also be reused after checking python --version.

Manual inventory demonstration

1. Open http://127.0.0.1:8000/admin/ and sign in with the account you created. 2. Add equipment with asset tag LAP-001, name Demo laptop, category Laptop, and Active selected. 3. Save, search for LAP-001, and edit its description. 4. Clear Active and save to retire the item. 5. Try adding another item with LAP-001 and confirm the duplicate is rejected. Use a fresh database or a different tag if LAP-001 already exists.

The expected result is a stored inventory record that can be retrieved and edited. Retirement changes the flag and preserves the record. This demonstrates inventory persistence, not reservation availability or checkout processing.

Running checks

```
# Stop runserver with Control+C before using this terminal.
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test inventory --verbosity 2
```

These commands assume you remain in src/campus_checkout with the virtual environment active. On another day, activate .venv from the repository root before changing into that directory. A running development server occupies its terminal; do not type test commands into its log output.

Recorded execution and evidence

The assistant reran the copied application on October 27, 2026. The system check reported no issues, the migration check reported no changes, fresh database migrations completed, and all six inventory tests passed. This is an assistant-run verification record, not a claim that the student independently reran these checks on October 27. The student previously showed the running inventory interface on October 17.

Unedited captured command results are under docs/step_2/week_1/verification/: system_check.txt, migration_check.txt, database_setup.txt, test_output.txt, and environment.txt. Each command log records its command and exit status. The verification database was excluded from the upload package.

What the tests verify

Database uniqueness: a duplicate asset tag raises IntegrityError. Input validation: an invalid tag fails model validation. Anonymous access: the equipment list redirects to login. Borrower access: a direct add request does not create an item. Staff access: staff without model permissions cannot open the add form. Administrator workflow: creation and editing persist, retirement clears Active, and a delete request is denied while the record remains.

Status against the original requirements

FR-01 - partially implemented. Django admin authentication is available. A borrower login interface and complete login/logout acceptance coverage remain incomplete.

FR-02 - partially implemented. Admin endpoints enforce authentication and model permissions. Borrower ownership rules and reservation authorization are not implemented.

FR-03 - partially implemented. Administrators can create and update inventory with unique identifiers. The inactive-item reservation restriction remains unimplemented because reservations are not integrated.

FR-04 - planned for borrowers. Admin inventory search exists, but borrower catalog access and time-interval availability checks do not.

FR-05 through FR-12 - planned in the application. The earlier standalone reservation proof of concept remains separate. It does not demonstrate integrated approval, cancellation, checkout, return, overdue reporting, or reservation/loan persistence in this application.

Design decisions and limitations

The implementation retains the Step 1 Django and SQLite architecture. Django admin provides the first inventory interface, allowing database and authorization behavior to be exercised before custom borrower pages exist. This is an implementation sequence decision, not a change from a software development project to a research project.

Only the Equipment domain model is implemented in this application. Reservations and checkouts from the full design remain future models. The standalone approval proof of concept has not been connected to this application. Deletion is disabled in admin; general database deletion protection for future loan relationships still needs design and implementation.

Configuration uses localhost hosts, DEBUG enabled, America/New_York, and Django CSRF middleware. DJANGO_SECRET_KEY can override the clearly labeled public development placeholder. These are development settings and must not be used for deployment. The package excludes the local SQLite database, virtual environment, caches, and real credentials. Test fixture credentials are synthetic and created only in the test database.

Week 2 implementation plan

Target a coherent subset of FR-02, FR-04, FR-05, and FR-06: add borrower catalog access and server-side permissions; define a Reservation model and validated request form; persist pending requests linked to the signed-in user; and connect administrator approval to a transaction-protected overlap check. Record each requirement as implemented or partial based on actual behavior, not planned behavior.

Add tests for valid requests, invalid time intervals, inactive equipment, unauthorized operations, and conflicting approvals. Keep initial SQLite transaction behavior under test when adapting the earlier proof of concept. Week 3 will integrate a borrower request-to-admin-decision workflow and verify persisted state and meaningful error handling.

Repository correction and provenance

Use docs/step_2/week_1/implementation_foundation.pdf, matching the assignment milestone table and existing repository path. Keep Step 1 submissions unchanged. A new Week 1 README identifies this report as the software development submission and marks research_foundation.md/pdf and their input-generation evidence as superseded. No earlier files need to be deleted or silently renamed.

This correction package was prepared on October 27. Preserve real commit dates and do not imply that this correction was uploaded in earlier weeks. Week 2 and Week 3 reports are not included as completed milestones. ChatGPT assisted with the application, tests, verification, and this report. The student should review the code, reproduce the checks, and explain the demonstrated behavior.